

Choosing the right design pattern is often more difficult than implementing it. In Python, the "right" pattern is usually the one that minimizes boilerplate and maximizes readability.

Below is a decision tree and selection guide categorized by the **intent** of your architectural problem.

1. The Pattern Selection Decision Tree

Ask yourself: **"What is my primary pain point right now?"**

A. "Object creation is becoming a mess." (Creational)

- Do I need a single instance shared everywhere? → **Singleton** (or a **Module-level variable**).
- Does the user need to create objects without knowing the specific class? → **Factory Method**.
- Do I need to build a complex object step-by-step? → **Builder**.
- Is creating new objects too "expensive" (RAM/CPU)? → **Prototype** (via `copy.deepcopy`).

B. "My classes are too tangled or complex." (Structural)

- Do I need to simplify a complex API for a user? → **Facade**.
- Do I need to make two incompatible interfaces work together? → **Adapter**.
- Do I need to separate my logic from the platform/driver implementation? → **Bridge**.
- Do I have millions of small objects eating up my RAM? → **Flyweight** (using `__slots__`).
- Do I need to treat a single object and a collection of objects the same way? → **Composite**.

C. "My logic is full of if/else and hardcoded steps." (Behavioral)

- Do I have massive if/else blocks checking for a 'status' or 'mode'? → **State**.
 - Do I want to swap algorithms at runtime (e.g., different sort methods)? → **Strategy**.
 - Do I need to notify multiple objects when something changes? → **Observer**.
 - Do I need to process a request through a series of filters/handlers? → **Chain of Responsibility**.
 - Do I need to save and restore the state of an object (Undo/Redo)? → **Memento**.
-

2. Comparison Table: Similar Patterns

Sometimes two patterns look identical in Python. Here is how to tell them apart:

Pattern A	Pattern B	How to choose?
Strategy	State	Strategy is usually set once by the user. State changes itself as the object evolves.
Proxy	Decorator	Proxy controls access/lifecycle of the object. Decorator adds new <i>functionality</i> to the object.
Facade	Adapter	Facade creates a <i>new, simpler</i> interface. Adapter makes an <i>existing</i> interface match a specific requirement.
Factory	Abstract Factory	Factory creates one type of object. Abstract Factory creates <i>families</i> of related objects.

3. The "Pythonic" Reality Check (Anti-Pattern Guide)

Before you implement a pattern, ask if Python's built-in features already solve the problem:

- **Don't use a Singleton class** if a simple Python module (mymodule.py) with a few variables will suffice.
 - **Don't use a Command class** if a list of lambda functions or function references is enough.
 - **Don't use an Iterator class** if you can just use the yield keyword to create a generator.
 - **Don't use a Strategy class** if you can just pass a function as an argument (First-class functions).
-